

Human-in-the-Loop Interface Design Challenge

Hands-On Learning Report — Module 3: Feedback-Aware Chat UI

1. Overview

I built a minimal chat interface for a customer-support assistant that makes answers transparent and gives users a low-friction way to push back. Every assistant message carries feedback controls (thumbs up, thumbs down, flag, explain why, ask to clarify). Each action sends a structured JSON payload to a backend that stores it and returns aggregate counts, which the UI shows in a Feedback Summary panel. The panel proves the loop is real: the numbers come from the server, not from local UI state.

Stack: React 18 + Vite, Vercel AI SDK v5 (ai 5.0.271, @ai-sdk/react 2.0.274, useChat with DefaultChatTransport), FastAPI backend with an in-memory feedback store.

Honest scope note: the chat route uses a deterministic **mock model** that streams canned answers (the lab allows a mock backend). If OPENAI_API_KEY is set, the route switches to an OpenAI streaming call; that path was not exercised here. Screenshots are real captures of the running app taken with a headless Chromium browser (Playwright) that drove the UI like a user.

2. Architecture and Endpoints

Route	Purpose
POST /api/chat	Receives the AI SDK message list and streams the reply as server-sent events in the AI SDK UI-message-stream format (start, text-delta, finish). Assigns each assistant message a stable id and counts it toward Total responses.
POST /api/feedback	Validates the payload (Pydantic) and upserts it in memory keyed by (sessionId, messageId), so changing a rating updates the record instead of double counting. Returns fresh session stats.
GET /api/feedback/stats	Returns aggregate counts for a session and overall: total responses, thumbs up/down, flagged, comments, feedback rate, satisfaction.

Feedback payload sent on every action:

```
{
  "messageId": "msg_6978de8e30",
  "rating": "down",           // "up" | "down" | null
  "flagged": true,
  "comment": "It says it is not certain - I need a definite answer for annual plans.",
  "timestamp": "2026-10-02T05:04:38.211Z",
  "sessionId": "c074081a-..." // random per page load
}
```

3. UI Design (Activity 1)

- **Clear separation of speakers:** user messages are right-aligned on a blue background; assistant messages are left-aligned on neutral gray. Each shows a small label and timestamp.

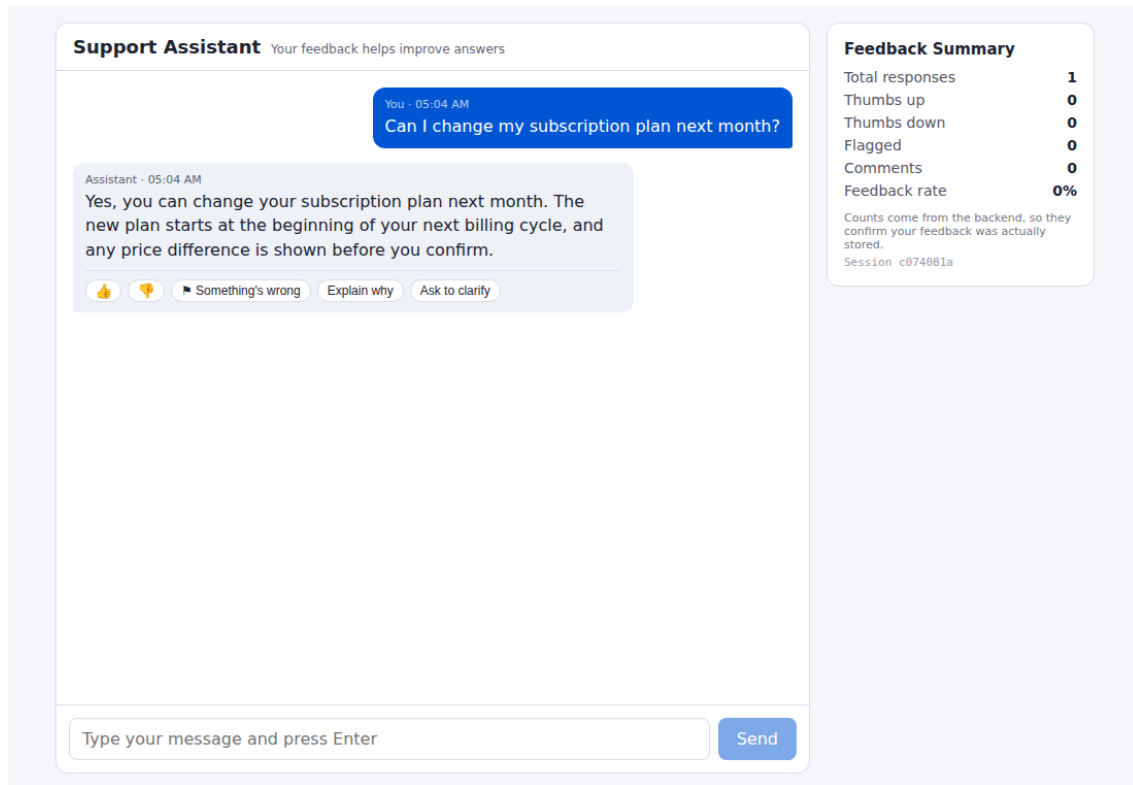
- **Streaming:** useChat renders tokens as they arrive with a blinking cursor; controls appear only after the answer finishes, so users rate a complete answer.
- **Input:** text box at the bottom; Enter or the Send button submits, the box clears, and it is disabled while a reply streams.
- **Stable ids:** every assistant message has the id assigned by the server stream (msg_XXXX), used to map feedback to the right message.

4. Feedback Controls and Metrics (Activity 2)

- **Thumbs up / down:** toggle buttons with aria-pressed and color state; clicking the active one clears the rating.
- **Flag (Something's wrong):** toggles to Flagged with an amber highlight.
- **Explain why:** opens a small text box; submitting sends the comment together with the current rating and flag.
- **Ask to clarify:** sends a follow-up user message asking the assistant to rephrase, so users can request clarification without leaving the chat.
- **Status text:** each action shows Saving..., then Feedback saved, or Try again on failure.
- **Feedback Summary panel:** Total responses, Thumbs up, Thumbs down, Flagged, Comments, Feedback rate, refreshed from the backend after every feedback action and after each reply.

5. Evidence

Screenshot 1: assistant answer with its feedback controls (before any feedback).



Screenshot 2: after interacting. Answer 1 rated thumbs up; answer 2 thumbs down, flagged and explained; a clarification was requested and answered. The summary panel shows backend counts.

Support Assistant

Your feedback helps improve answers

Assistant - 05:04 AM

Yes, you can change your subscription plan next month. The new plan starts at the beginning of your next billing cycle, and any price difference is shown before you confirm.

👍

👎

Something's wrong

Explain why

Ask to clarify

Feedback saved

You - 05:04 AM

Can I get a refund?

Assistant - 05:04 AM

I believe refunds are available within 30 days of purchase, but I'm not fully certain this applies to annual plans. Please confirm with support before relying on it.

👍

👎

Flagged

Explain why

Ask to clarify

Feedback saved

You - 05:04 AM

Can you clarify that last answer?

Assistant - 05:04 AM

Sure, let me put it more simply: plan changes are applied at the start of your next billing cycle, so you will not be charged twice in the same month.

👍

👎

Something's wrong

Explain why

Ask to clarify

Type your message and press Enter

Send

Feedback Summary

Total responses

3

Thumbs up

1

Thumbs down

1

Flagged

1

Comments

1

Feedback rate

67%

Counts come from the backend, so they confirm your feedback was actually stored.

Session c074081a

Human-in-the-Loop Interface Design Challenge - Hands-On Learning Report

Page 4

Backend log and stats from the same run

```
[05:04:36] feedback session=c074081a msg=msg_09d8f58426 rating=up   flagged=False comment=None
[05:04:37] feedback session=c074081a msg=msg_6978de8e30 rating=down flagged=False comment=None
[05:04:37] feedback session=c074081a msg=msg_6978de8e30 rating=down flagged=True  comment=None
[05:04:38] feedback session=c074081a msg=msg_6978de8e30 rating=down flagged=True  comment='It says it is not certain - I
                                                    answer for annual plans.'
```

```
GET /api/feedback/stats ->
{ total_responses: 3, thumbs_up: 1, thumbs_down: 1, flagged: 1, comments: 1,
  feedback_rate: 0.67, satisfaction: 0.5 }
```

The log shows that the thumbs-down, flag and comment on the same message update one record (three events, one stored entry), so the counts stay accurate. The panel values (3 / 1 / 1 / 1 / 67%) match the backend stats.

6. Reflection on UX Choices

Where I placed the controls and why

The controls sit directly under each assistant message, inside the same bubble, separated by a thin rule. Proximity makes it unambiguous which answer is being rated, and the user does not have to move far from the text they just read. They appear only after streaming ends, so feedback targets the complete answer.

Low-friction rating and flagging

Rating is one click, with familiar thumbs icons plus tooltips. Flagging is a separate, clearly labeled one-click action, so a user who thinks an answer is wrong does not need to write anything. Writing a reason is optional and tucked behind Explain why, which keeps the common path fast while allowing richer feedback. Buttons toggle, so a mistaken click is easy to undo, and each action shows immediate status text so the user knows it registered.

How visible metrics build trust

The Feedback Summary panel shows counts pulled from the server after each action. Seeing Thumbs up and Flagged tick up immediately gives concrete proof that the system listens, instead of a feedback button that silently does nothing. The Feedback saved confirmation covers the individual action; the panel covers the aggregate. The refund answer, where the assistant honestly said it was unsure, also showed that admitting uncertainty plus an easy flag path lets users correct the system when it hedges.

What I would add next

- **Confidence indicators** (for example a low/medium/high chip) so uncertain answers look different from confident ones.
- **A Why this answer? section** showing the sources or policy snippets behind the answer, so users can verify it.
- **Structured reasons** on thumbs-down (inaccurate, outdated, unclear, unsafe) as quick-pick chips before free text.
- **Visible follow-through**, such as a note that a previous flag led to a fix, to close the loop beyond counters.

- **Persistence and privacy:** a database instead of memory, user-level controls over stored comments, and rate limiting on the feedback endpoint.
- **Usability testing** with real users, including keyboard and screen-reader checks (aria-pressed and live regions are already in place).

7. Source Code

backend/main.py

```
"""
HITL feedback lab backend (FastAPI)
  POST /api/chat      -> streams an assistant reply in the Vercel AI SDK UI-message-stream format
  POST /api/feedback  -> stores/updates feedback for one assistant message, returns fresh stats
  GET  /api/feedback/stats -> aggregate counts (overall + per session)
Feedback lives in a simple in-memory dict (lost on restart) - fine for the lab.
Without OPENAI_API_KEY the chat route uses a deterministic mock model.
"""

import asyncio, json, os, uuid
from datetime import datetime, timezone
from pathlib import Path
from typing import Literal, Optional

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import StreamingResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field

app = FastAPI(title="HITL Feedback Lab")
app.add_middleware(CORSMiddleware, allow_origins=["http://localhost:5173"],
                  allow_methods=["*"], allow_headers=["*"])

# ----- in-memory stores
RESPONSES: dict[str, set] = {}      # sessionId -> assistant message ids generated
FEEDBACK: dict[tuple, dict] = {}    # (sessionId, messageId) -> latest feedback record

# ----- mock model
def mock_answer(q: str) -> str:
    q = q.lower()
    if "clarif" in q:
        return ("Sure, let me put it more simply: plan changes are applied at the start "
                "of your next billing cycle, so you will not be charged twice in the same month.")
    if "refund" in q:
        return ("I believe refunds are available within 30 days of purchase, but I'm not fully "
                "certain this applies to annual plans. Please confirm with support before relying on it.")
    if "invoice" in q or "billing" in q:
        return ("Invoices are generated on the 1st of each month and emailed to the account owner. "
                "You can also download past invoices from Settings > Billing.")
    if "cancel" in q:
        return ("You can cancel anytime from Settings > Subscription. Your access continues "
                "until the end of the current billing period.")
    if "plan" in q or "subscription" in q or "upgrade" in q:
        return ("Yes, you can change your subscription plan next month. The new plan starts at the "
                "beginning of your next billing cycle, and any price difference is shown before you confirm.")
    return ("I'm not sure I understood that. Could you tell me whether your question is about "
            "plans, billing, invoices, or cancellations?")

async def openai_stream(q: str):
    from openai import AsyncOpenAI      # optional path, used only if OPENAI_API_KEY is set
    client = AsyncOpenAI()
    stream = await client.chat.completions.create(
        model=os.getenv("LLM_MODEL", "gpt-4o-mini"), stream=True,
        messages=[{"role": "system", "content": "You are a concise support assistant."},
                  {"role": "user", "content": q}])
    async for chunk in stream:
        if chunk.choices and chunk.choices[0].delta.content:
            yield chunk.choices[0].delta.content

async def mock_stream(q: str):
    for word in mock_answer(q).split(" "):
        yield word + " "
        await asyncio.sleep(0.035)      # simulate token streaming

def sse(obj) -> str:
    return f"data: {json.dumps(obj)}\n\n"

@app.post("/api/chat")
async def chat(request: Request):
    body = await request.json()
    session_id = body.get("sessionId", "anon")
    last_user = next((m for m in reversed(body.get("messages", [])) if m.get("role") == "user"), {})
    text = "".join(p.get("text", "") for p in last_user.get("parts", []) if p.get("type") == "text")
```

```

msg_id, text_id = "msg_" + uuid.uuid4().hex[:10], "txt_" + uuid.uuid4().hex[:6]
RESPONSES.setdefault(session_id, set()).add(msg_id) # counts toward "Total responses"
tokens = openai_stream(text) if os.getenv("OPENAI_API_KEY") else mock_stream(text)

async def gen():
    yield sse({"type": "start", "messageId": msg_id})
    yield sse({"type": "text-start", "id": text_id})
    async for t in tokens:
        yield sse({"type": "text-delta", "id": text_id, "delta": t})
    yield sse({"type": "text-end", "id": text_id})
    yield sse({"type": "finish"})
    yield "data: [DONE]\n\n"

return StreamingResponse(gen(), media_type="text/event-stream",
    headers={"x-vercel-ai-ui-message-stream": "v1", "Cache-Control": "no-cache", "x-accel-buffering": "no"})

# ----- feedback
class Feedback(BaseModel):
    messageId: str
    rating: Optional[Literal["up", "down"]] = None
    flagged: bool = False
    comment: Optional[str] = Field(default=None, max_length=1000)
    timestamp: str
    sessionId: str

def stats_for(session_id: Optional[str] = None) -> dict:
    recs = [r for (s, _), r in FEEDBACK.items() if session_id in (None, s)]
    total = sum(len(v) for k, v in RESPONSES.items() if session_id in (None, k))
    rated = sum(1 for r in recs if r["rating"])
    touched = sum(1 for r in recs if r["rating"] or r["flagged"] or r["comment"])
    return {"total_responses": total,
        "thumbs_up": sum(1 for r in recs if r["rating"] == "up"),
        "thumbs_down": sum(1 for r in recs if r["rating"] == "down"),
        "flagged": sum(1 for r in recs if r["flagged"]),
        "comments": sum(1 for r in recs if r["comment"]),
        "feedback_rate": round(touched / total, 2) if total else 0.0,
        "satisfaction": round(sum(1 for r in recs if r["rating"] == "up") / rated, 2) if rated else None}

@app.post("/api/feedback")
def post_feedback(fb: Feedback):
    # keyed by (session, message): changing your mind updates the record instead of double counting
    FEEDBACK[{fb.sessionId, fb.messageId}] = fb.model_dump()
    print(f"[{datetime.now(timezone.utc).%H:%M:%S}] feedback session={fb.sessionId[:8]} msg={fb.messageId} "
        f"rating={fb.rating} flagged={fb.flagged} comment={fb.comment!r}", flush=True)
    return {"ok": True, "stats": stats_for(fb.sessionId)}

@app.get("/api/feedback/stats")
def get_stats(sessionId: Optional[str] = None):
    return {"session": stats_for(sessionId) if sessionId else None, "overall": stats_for()}

# serve the built React app (production-style single server) if it exists
DIST = Path(__file__).resolve().parent.parent / "frontend" / "dist"
if DIST.exists():
    app.mount("/", StaticFiles(directory=DIST, html=True), name="web")

```

frontend/src/App.jsx

```
import { useEffect, useRef, useState } from 'react';
import { useChat } from '@ai-sdk/react';
import { DefaultChatTransport } from 'ai';

// One random session id per page load, so different runs can be told apart on the backend.
const SESSION_ID = crypto.randomUUID();
const transport = new DefaultChatTransport({ api: '/api/chat', body: { sessionId: SESSION_ID } });

const textOf = (m) => m.parts.filter((p) => p.type === 'text').map((p) => p.text).join('');
const timeOf = (d) => d.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });

export default function App() {
  const { messages, sendMessage, status } = useChat({ transport });
  const [input, setInput] = useState('');
  const [fb, setFb] = useState({}); // messageId -> { rating, flagged, comment, note, explainOpen, draft }
  const [stats, setStats] = useState(null); // session stats from the backend
  const stamps = useRef({}); // messageId -> Date first seen
  const bottom = useRef(null);

  const busy = status === 'submitted' || status === 'streaming';
  messages.forEach((m) => { stamps.current[m.id] ??= new Date(); });

  const loadStats = () =>
    fetch(`/api/feedback/stats?sessionId=${SESSION_ID}`).then((r) => r.json()).then((d) => setStats(d.session)).catch(() => {});

  useEffect(() => { loadStats(); }, []);
  useEffect(() => { if (status === 'ready') loadStats(); }, [status]);
  useEffect(() => { bottom.current?.scrollIntoView({ behavior: 'smooth' }); }, [messages]);

  const patch = (id, p) => setFb((s) => ({ ...s, [id]: { ...s[id], ...p } }));

  // Sends the FULL feedback state for one message (structured payload per the lab spec).
  async function submitFeedback(id, next) {
    const cur = { rating: null, flagged: false, comment: '', ...fb[id], ...next };
    patch(id, { ...next, note: 'Saving...' });
    try {
      const res = await fetch('/api/feedback', {
        method: 'POST', headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          messageId: id, rating: cur.rating, flagged: cur.flagged,
          comment: cur.comment || null, timestamp: new Date().toISOString(), sessionId: SESSION_ID,
        }),
      });
      if (!res.ok) throw new Error('bad status');
      const data = await res.json();
      setStats(data.stats);
      patch(id, { note: 'Feedback saved' });
    } catch {
      patch(id, { note: 'Try again' });
    }
  }

  const rate = (id, value) => submitFeedback(id, { rating: fb[id]?.rating === value ? null : value });
  const flag = (id) => submitFeedback(id, { flagged: !fb[id]?.flagged });
  const send = (text) => { if (text.trim() && !busy) { sendMessage({ text }); setInput(''); } };

  return (
    <div className="app">
      <main className="chat">
        <header><h1>Support Assistant</h1><span className="sub">Your feedback helps improve answers</span></header>

        <div className="list" role="log" aria-live="polite">
          {messages.length === 0 && (
            <div className="empty">
              Ask something, for example:
              <div className="chips">
                {['Can I change my subscription plan next month?', 'Can I get a refund?', 'Where do I find my invoice?'].map((q) => (
                  <button key={q} className="chip" onClick={() => send(q)}>{q}</button>))}
              </div>
            </div>
          )}

          {messages.map((m, i) => {
            const mine = m.role === 'user';
            const f = fb[m.id] || {};
            const streaming = !mine && busy && i === messages.length - 1;
            return (
              <div key={m.id} className={`row ${mine ? 'user' : 'bot'}`}>

```

```

<div className="bubble">
  <div className="meta">{mine ? 'You' : 'Assistant'} · {timeOf(stamps.current[m.id])}</div>
  <div className="text">{textOf(m)}{streaming && <span className="cursor">■</span></div>

  {!mine && !streaming && (
    <div className="fb" aria-label="Feedback controls">
      <button className={`ic ${f.rating === 'up' ? 'on up' : ''}`} aria-pressed={f.rating === 'up'}
        title="Helpful answer" aria-label="Thumbs up" onClick={() => rate(m.id, 'up')}>■</button>
      <button className={`ic ${f.rating === 'down' ? 'on down' : ''}`} aria-pressed={f.rating === 'down'}
        title="Not helpful" aria-label="Thumbs down" onClick={() => rate(m.id, 'down')}>■</button>
      <button className={`txt ${f.flagged ? 'on flag' : ''}`} aria-pressed={!f.flagged}
        title="Report a wrong or unsafe answer" onClick={() => flag(m.id)}>■ {f.flagged ? 'Flagged' : "Something's"}
      <button className="txt" title="Tell us what felt wrong" onClick={() => patch(m.id, { explainOpen: !f.explainOpen })}
      <button className="txt" title="Ask the assistant to rephrase" disabled={busy}
        onClick={() => send('Can you clarify that last answer?')}>Ask to clarify</button>
      {f.note && <span className={`note ${f.note === 'Try again' ? 'err' : ''}`} role="status">{f.note}</span>}
    </div>)}

    {!mine && f.explainOpen && (
      <div className="explain">
        <textarea rows={2} maxLength={1000} placeholder="What was wrong or unclear?" value={f.draft ?? f.comment ?? ''}
          onChange={(e) => patch(m.id, { draft: e.target.value })} />
        <button className="send small" onClick={() => { submitFeedback(m.id, { comment: f.draft ?? '' }); patch(m.id, { exp
      </div>)}
    </div>);
  }}
  <div ref={bottom} />
</div>

<div className="composer">
  <input value={input} placeholder="Type your message and press Enter" disabled={busy}
    onChange={(e) => setInput(e.target.value)} onKeyDown={(e) => e.key === 'Enter' && send(input)} />
  <button className="send" onClick={() => send(input)} disabled={busy || !input.trim()}>Send</button>
</div>
</main>

<aside className="panel" aria-label="Feedback summary">
  <h2>Feedback Summary</h2>
  <dl>
    <dt>Total responses</dt><dd id="m-total">{stats?.total_responses ?? 0}</dd>
    <dt>Thumbs up</dt><dd id="m-up">{stats?.thumbs_up ?? 0}</dd>
    <dt>Thumbs down</dt><dd id="m-down">{stats?.thumbs_down ?? 0}</dd>
    <dt>Flagged</dt><dd id="m-flag">{stats?.flagged ?? 0}</dd>
    <dt>Comments</dt><dd>{stats?.comments ?? 0}</dd>
    <dt>Feedback rate</dt><dd>{Math.round((stats?.feedback_rate ?? 0) * 100)}%</dd>
  </dl>
  <p className="hint">Counts come from the backend, so they confirm your feedback was actually stored.</p>
  <p className="sid">Session {SESSION_ID.slice(0, 8)}</p>
</aside>
</div>
  );
}

```

frontend/src/styles.css

```
:root { --blue:#0056d2; --bg:#f4f6fa; --line:#d9dee7; --text:#1d2433; }
* { box-sizing: border-box; }
body { margin:0; font-family: system-ui, -apple-system, "Segoe UI", sans-serif; background:var(--bg); color:var(--text); }
.app { display:grid; grid-template-columns: 1fr 260px; gap:16px; max-width:1040px; margin:0 auto; padding:16px; height:100vh; }
.chat { display:flex; flex-direction:column; background:#fff; border:1px solid var(--line); border-radius:12px; min-height:0; }
header { padding:12px 16px; border-bottom:1px solid var(--line); display:flex; align-items:baseline; gap:10px; }
h1 { font-size:18px; margin:0; } .sub { color:#667; font-size:12px; }
.list { flex:1; overflow-y:auto; padding:16px; display:flex; flex-direction:column; gap:14px; }
.row { display:flex; } .row.user { justify-content:flex-end; }
.bubble { max-width:78%; padding:10px 12px; border-radius:12px; }
.user.bubble { background:var(--blue); color:#fff; border-bottom-right-radius:3px; }
.bot.bubble { background:#eef1f6; border-bottom-left-radius:3px; }
.meta { font-size:11px; opacity:.7; margin-bottom:3px; } .text { line-height:1.45; white-space:pre-wrap; }
.cursor { animation: blink 1s steps(2) infinite; } @keyframes blink { 50% { opacity:0; } }
.fb { display:flex; flex-wrap:wrap; gap:6px; align-items:center; margin-top:8px; padding-top:8px; border-top:1px solid #dde2ea; }
.fb button { cursor:pointer; border:1px solid var(--line); background:#fff; border-radius:999px; font-size:12px; padding:3px 9px; color:var(--text); }
.fb button.ic { font-size:14px; padding:2px 8px; } .fb button:hover:not(:disabled) { border-color:var(--blue); }
.fb button:disabled { opacity:.5; cursor:not-allowed; }
.fb .on.up { background:#e3f6ea; border-color:#2e9e5b; } .fb .on.down { background:#fdeaea; border-color:#d04a4a; }
.fb .on.flag { background:#fff2d9; border-color:#d98a00; }
.note { font-size:11px; color:#2e7d4f; margin-left:4px; } .note.err { color:#c0392b; }
.explain { display:flex; gap:8px; margin-top:8px; } .explain textarea { flex:1; font:inherit; font-size:13px; padding:6px; border:1px solid var(--line); }
.composer { display:flex; gap:8px; padding:12px; border-top:1px solid var(--line); }
composer input { flex:1; font:inherit; padding:10px 12px; border:1px solid var(--line); border-radius:8px; }
.send { background:var(--blue); color:#fff; border:0; border-radius:8px; padding:0 18px; font:inherit; cursor:pointer; } .send:disabled { opacity:.5; }
.empty { color:#667; font-size:14px; margin:auto; text-align:center; } .chips { display:flex; flex-direction:column; gap:8px; margin-top:8px; }
.chip { border:1px solid var(--line); background:#fff; border-radius:999px; padding:8px 14px; cursor:pointer; font:inherit; font-size:13px; }
.panel { background:#fff; border:1px solid var(--line); border-radius:12px; padding:16px; align-self:start; }
.panel h2 { font-size:15px; margin:0 0 10px 0; } dl { display:grid; grid-template-columns:1fr auto; gap:6px 10px; margin:0; font-size:14px; }
dt { color:#556; } dd { margin:0; font-weight:700; text-align:right; font-variant-numeric: tabular-nums; }
.hint { font-size:11px; color:#778; margin:12px 0 4px; } .sid { font-size:11px; color:#99a; margin:0; font-family:monospace; }
@media (max-width:760px) { .app { grid-template-columns:1fr; height:auto; } .panel { order:-1; } }
```